

Timer/Counter:

Timer와 Counter는 펄스 신호를 입력받아 셈하는 장치이며, MCU 내부의 클럭을 펄스로 입력받으면 타이머, 그 외의 외부 펄스 신호를 받으면 카운터로 분류한다.

타이머와 카운터는 펄스 신호를 카운트하며, 이를 이용하여 일정 간격으로 이벤트를 발생시키는 등으로 사용할 수 있다.

Timer0/2:

Atmega128에 내장된 비트 카운터로, MCU 내부 클럭 또는 외부 신호를 입력받아 카운트한다. 카운트한 값은 TCNT0/2에 저장되며, 최대값인 255를 넘게 되면 오버플로우 인터럽트를 발생시킬 수도 있다.

Timer0/2의 레지스터로는 TCCR, TCNT, OCR, TIMSK, TIFR이 있다.

TCCR0/2의 값을 통해 파형 생성 모드, 프리스케일러, 출력 모드 등을 설정할 수 있다.

TCNT0/2은 8비트 카운터의 값을 저장한다.

OCR0/2의 값을 설정하면 비교 기준값을 변경하여, 타이머가 초기화되는 값을 설정할 수 있다.

TIMSK의 TOIE0/2를 변경하면 Timer0/2의 오버플로우 인터럽트를 활성화 할 수 있다.

Counter의 원리:

카운터는 플립플롭 논리회로를 여러개 이은 형태로, 앞쪽의 플립플롭의 출력을 다음 플립플롭에서 클럭 신호로 사용한다. 각 플립플롭은 0 또는 1을 출력으로 가지므로 n개의 플립플롭을 이러한 형태로 연결할 때, 카운터는 $0 \sim 2^n - 1$ 까지의 수를 카운트할 수 있고, n비트 카운터라고 부른다.

*플립플롭은 클럭 신호를 받을 때마다 1비트 데이터를 변화시키는 논리회로이다.

Atmega128 사용 예제:

1) 1ms마다 동작 실행하기

```
#include <avr/io.h>
```

```
#include <avr/interrupt.h>
```

```
int main(void) {
```

```
    TCCR0 = (1 << WGM01);           // CTC 모드 설정
    OCR0  = 124;                     // 16MHz, 128분주 시 약 1ms 주기
    TCCR0 |= (1 << CS02) | (1 << CS00); // 128분주 프리스케일러
    TIMSK |= (1 << OCIE0);           // 비교일치 인터럽트 허용
    sei();
```

```
    while (1) {
        // 메인 루프
    }
```

```
}
```

```
ISR(TIMER0_COMP_vect) {  
    // 1ms마다 실행될 코드  
}
```

2) PWM 신호 출력 예제

```
#include <avr/io.h>
```

```
void PWM_Init(void) {
```

```
}
```

```
int main(void) {
```

```
    DDRB |= (1 << PB4);           // OC0 핀 출력 설정  
    TCCR0 = (1 << WGM01) | (1 << WGM00); // Fast PWM 모드  
    TCCR0 |= (1 << COM01);         // 비반전 모드 (일치 시 Low)  
    TCCR0 |= (1 << CS01);          // 8분주 프리스케일러  
    OCR0 = 128;                    // 듀티 비 50%
```

```
    while (1) {  
        // OC0(PB4) 핀에서 PWM 신호 출력  
    }
```

```
}
```